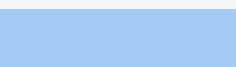
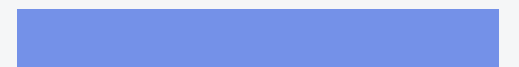
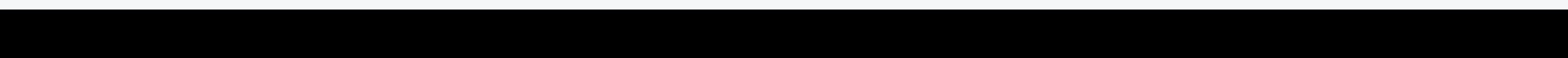
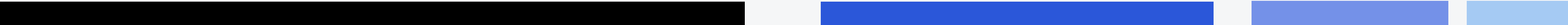


Lecture-9

Inheritance

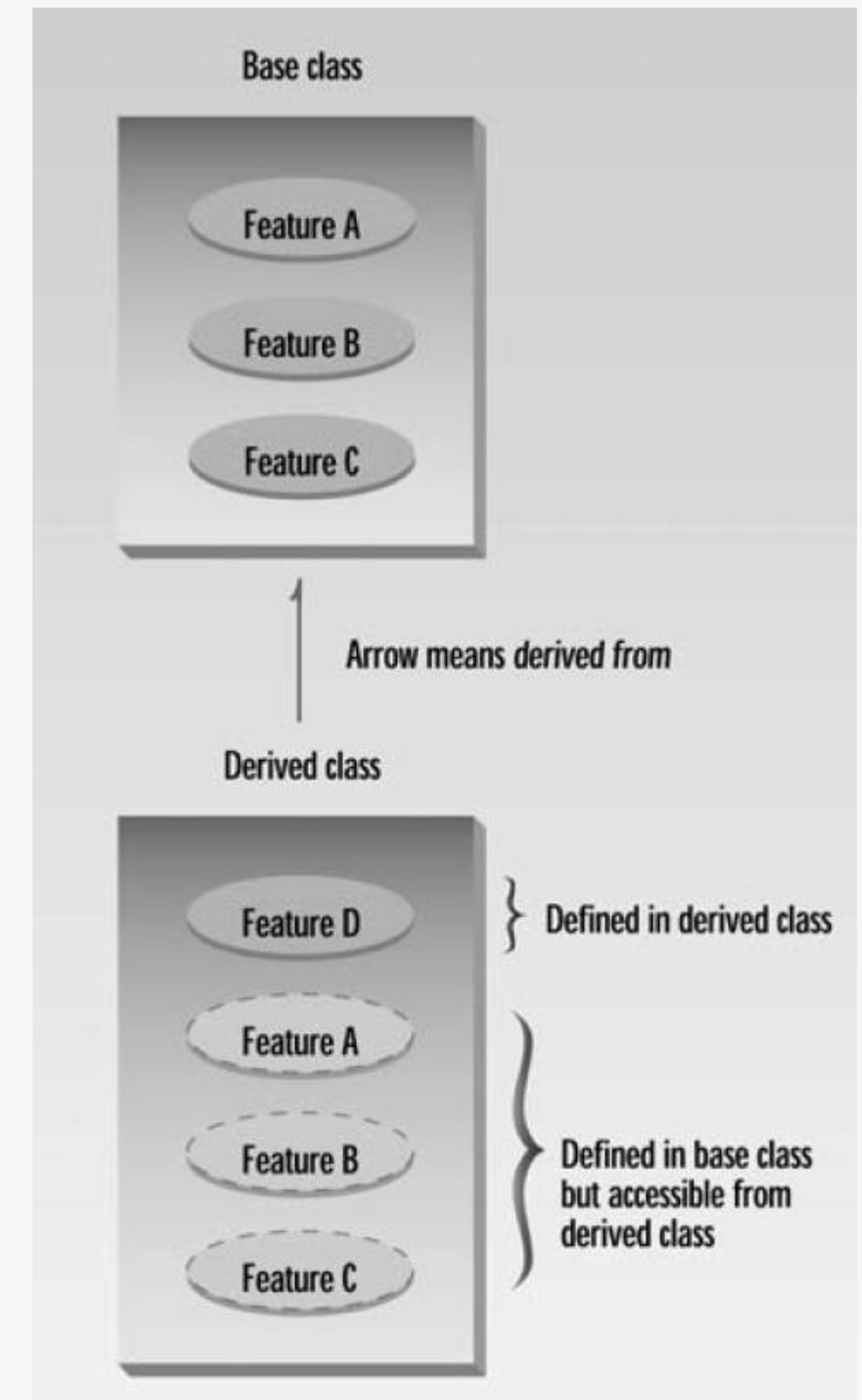


Introduction



Inheritance

- **Inheritance** is the process of creating new classes, called **derived classes**, from existing or base classes.
- The derived class inherits all the capabilities of the **base class** but can add embellishments and refinements of its own.
- The base class is unchanged by this process.



Inheritance

```
#include <iostream>
using namespace std;
class Counter
{
protected:
    unsigned int count;
public:
    Counter() : count(0)
    { }
    Counter(int c) : count(c)
    { }
    unsigned int get_count() const
    { return count; }
    void operator ++ ()
    { ++count; }
};
```

Inheritance

```
class CountDn : public Counter
{
public:
    void operator -- ()
    { --count; }
};
int main()
{
    CountDn c1;
    cout << "\nc1=" << c1.get_count();
    ++c1; ++c1; ++c1;
    cout << "\nc1=" << c1.get_count();
    --c1; --c1;
    cout << "\nc1=" << c1.get_count();
    cout << endl;
    return 0;
}
```


Inheritance

- ***Inheritance doesn't work in reverse.***
- The base class and its objects don't know anything about derived classes.
- In this example that means that objects of class Counter can't use the operator--() function in CountDn.
- If you want a counter that you can decrement, it must be of class CountDn, not Counter.

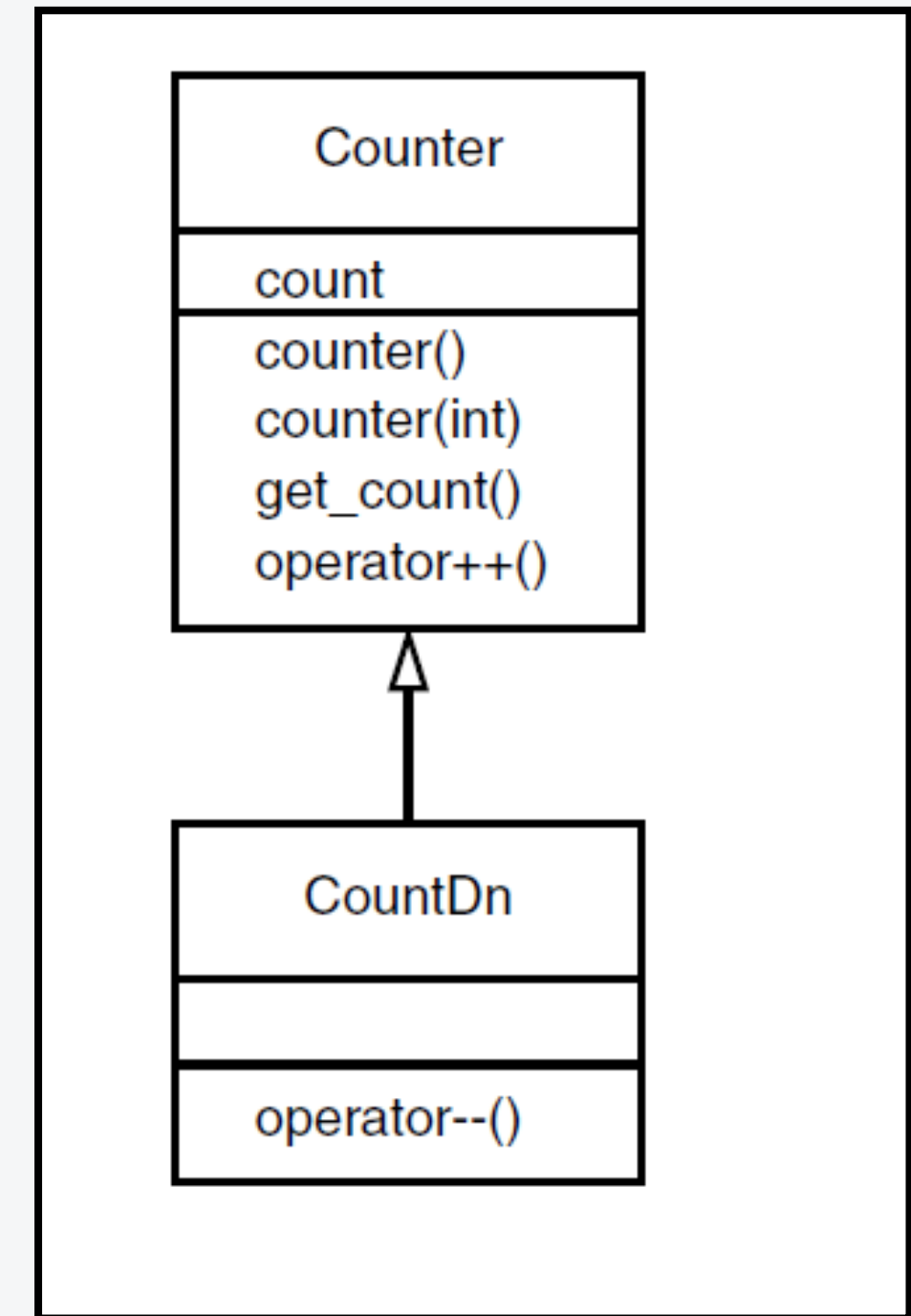
Inheritance & Accessibility

TABLE 9.1 Inheritance and Accessibility

<i>Access Specifier</i>	<i>Accessible from Own Class</i>	<i>Accessible from Derived Class</i>	<i>Accessible from Objects Outside Class</i>
public	yes	yes	yes
protected	yes	yes	no
private	yes	no	no

Inheritance & UML

- In the UML, **INHERITANCE** is called **GENERALIZATION**, because the parent class is a more general form of the child class.
- In UML class diagrams, generalization is indicated by a triangular arrowhead on the line connecting the parent and child classes.



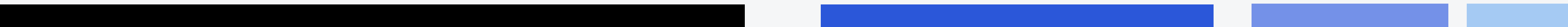
Accessing Data & Functions in Derived Class

```
#include <iostream>
using namespace std;
class Base {
public:
int x;
void setValue(int val) {
x = val;}
void showValue() {
cout << "Value of x (from Base): " << x << endl;}
};
```

Accessing Data & Functions in Derived Class

```
class Derived : public Base {  
public:  
void display() {  
cout << "Accessing x in Derived: " << x << endl;  
showValue();}  
};  
int main() {  
Derived obj;  
obj.setValue(10);  
obj.display();  
return 0;}
```

Constructors Inheritance



Inheritance & Constructors

- **Case-1 (Base class has a no arg constructor and derived class has no constructor)**

```
class Base
{
public:
Base():x(0) {cout << "Base constructor: " << x << endl;}
};
class Derived : public Base {
};
```

Inheritance & Constructors

- **Case-2 (Base class has a no arg constructor and derived class a no arg constructor)**

```
class Base
{
public:
Base():x(0) {cout << "Base constructor: " << x << endl;}
};

class Derived : public Base {
public:
Derived() : {cout << "Derived constructor\n";    }
};
```


Inheritance & Constructors

- **Case-3 (Base class has arg constructor and derived class a no arg constructor)**

```
class Base
{
public:
Base(int x) {cout << "Base constructor: " << x << endl;}
};
class Derived : public Base
{
public:
Derived() : Base(10) {cout << "Derived constructor\n";}
};
```

Inheritance & Constructors

- **Case-4 (Base class has only no arg constructor and derived class has initializer list)**

```
class Base
{public:
Base() {cout << "Base default constructor called\n";}
};

class Derived : public Base
{private:
    int value;
public:
    Derived(int v) : value(v) {
        cout << "Derived constructor called, value = " << value << endl;}
};
```

Inheritance & Constructors

- **Case-5**

```
class Base {  
public:  
int x;  
Base() {  
x = 0;  
cout << "Base default constructor (x = " << x << ")\n";}  
Base(int val) {  
x = val;  
cout << "Base parameterized constructor (x = " << x << ")\n";  
}  
};
```


Inheritance & Constructors

- **Case-5**

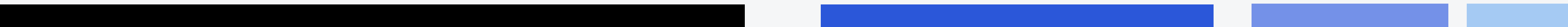
```
class Derived : public Base {  
public:  
    int y;  
    Derived() {  
        y = 0;  
        cout << "Derived default constructor (y = " << y << ")\n";}  
    Derived(int b) : y(b) {  
        cout << "Derived one-param constructor (y = " << y << ")\n";}  
    Derived(int a, int b) : Base(a), y(b) {  
        cout << "Derived two-param constructor (y = " << y << ")\n";}  
    };  
};
```

Inheritance & Constructors

- **Case-5**

```
int main() {  
    cout << "\n--- Case 1: Default Derived ---\n";  
    Derived d1;  
    cout << "\n--- Case 2: One-parameter Derived ---\n";  
    Derived d2(50);  
    cout << "\n--- Case 3: Two-parameter Derived ---\n";  
    Derived d3(10, 20);  
    return 0;  
}
```

Function Overriding



Overriding Member Functions

- Member functions in a derived class can override member functions of the base class.
- **Condition:** Both should have same name.

Overriding Member Functions

```
#include <iostream>
using namespace std;
class Base {
public:
void show() {
    cout << "Base class show() function\n";}
};
```

```
class Derived : public Base {
Public:
void show() {
    cout << "Derived class show() function\n";}
};

int main() {
Derived obj;
obj.show();
obj.Base::show();
cout << endl; return 0;}
```


Overriding Member Functions

```
#include <iostream>
using namespace std;
class Base {
public:
void show() {
    cout << "Base class show() function\n";}
};
```

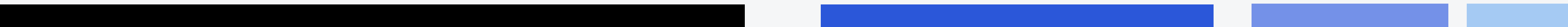
```
class Derived : public Base {
Public:
void show() {
    cout << "Derived class show() function\n";}
void display() {
    cout << "Calling Derived version:\n";
    show();
    Derived::show()
    cout << "Calling Base version using scope
        resolution:\n";

    Base::show();
    }
};
```

Overriding Member Functions

```
int main() {  
    Derived obj;  
    obj.show();  
    cout << endl;  
    obj.display();  
    return 0;}
```

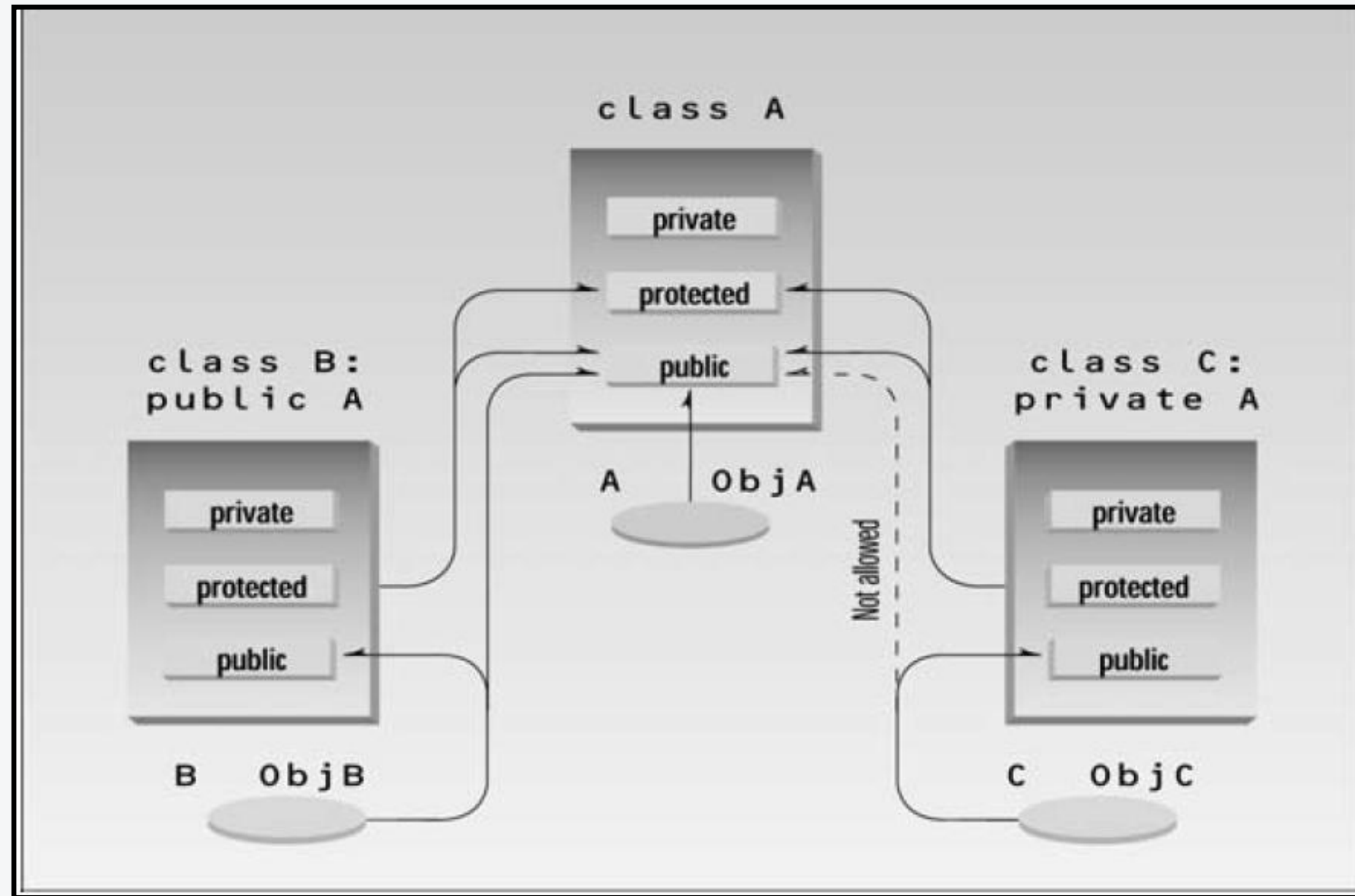
Public & Private Inheritance



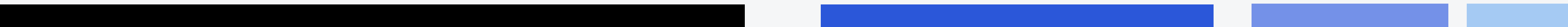
Public and Private Inheritance

- The keyword **public** specifies that objects of the derived class are able to access public member functions of the base class.
- The keyword **private** specifies that object of the derived class cannot access public member functions of the base class.

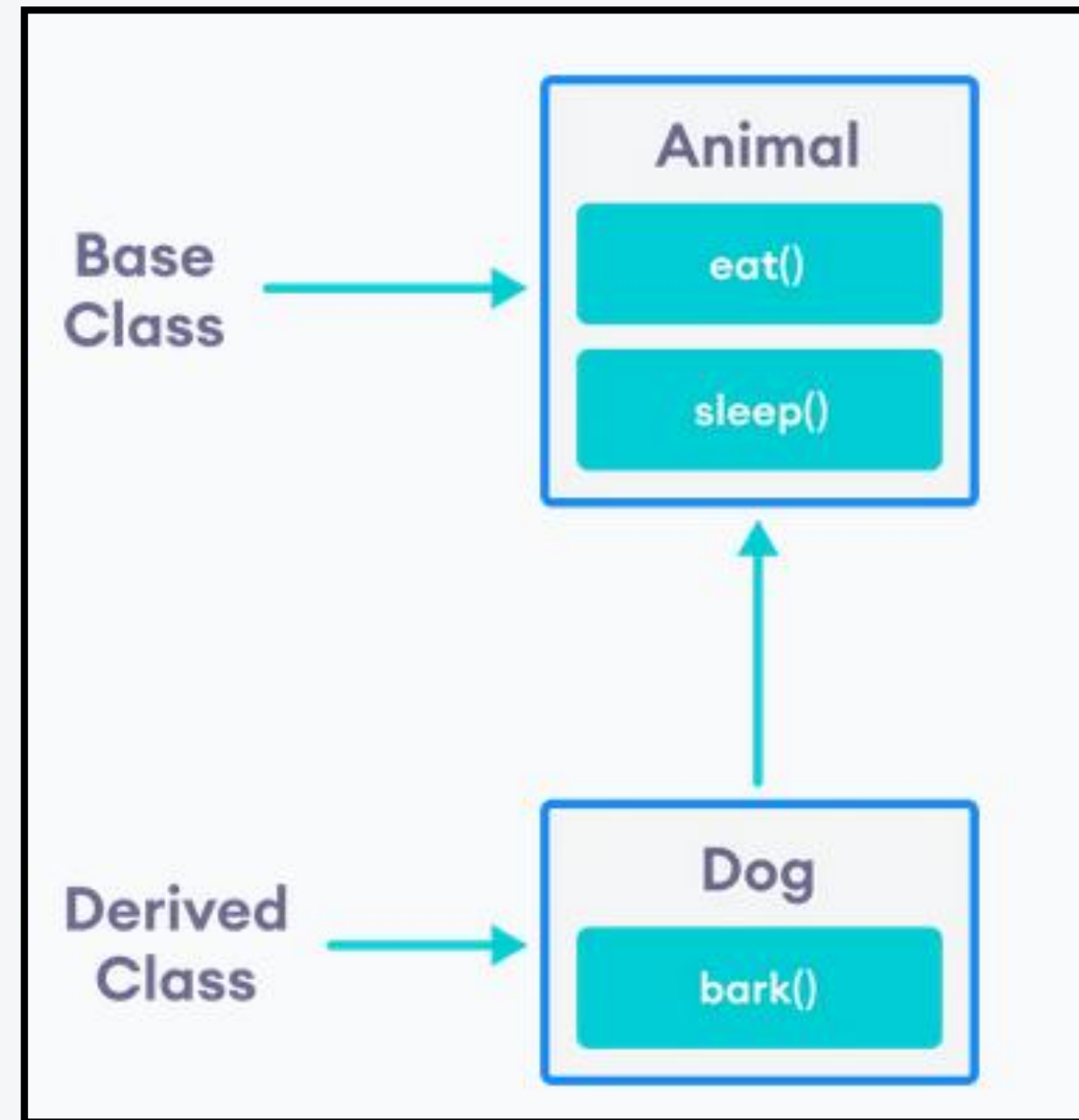
Public and Private Inheritance



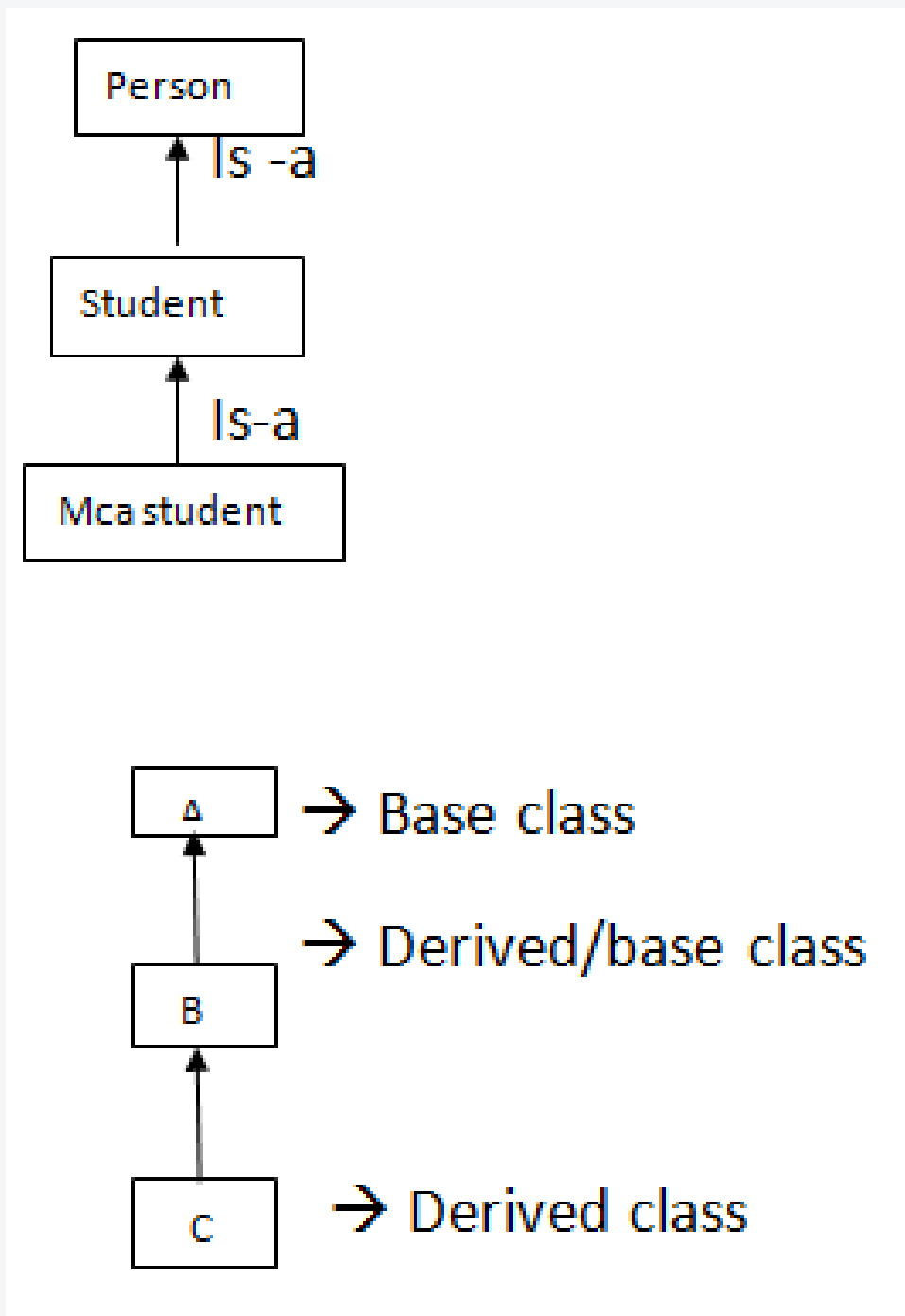
Levels of Inheritance



Single Inheritance

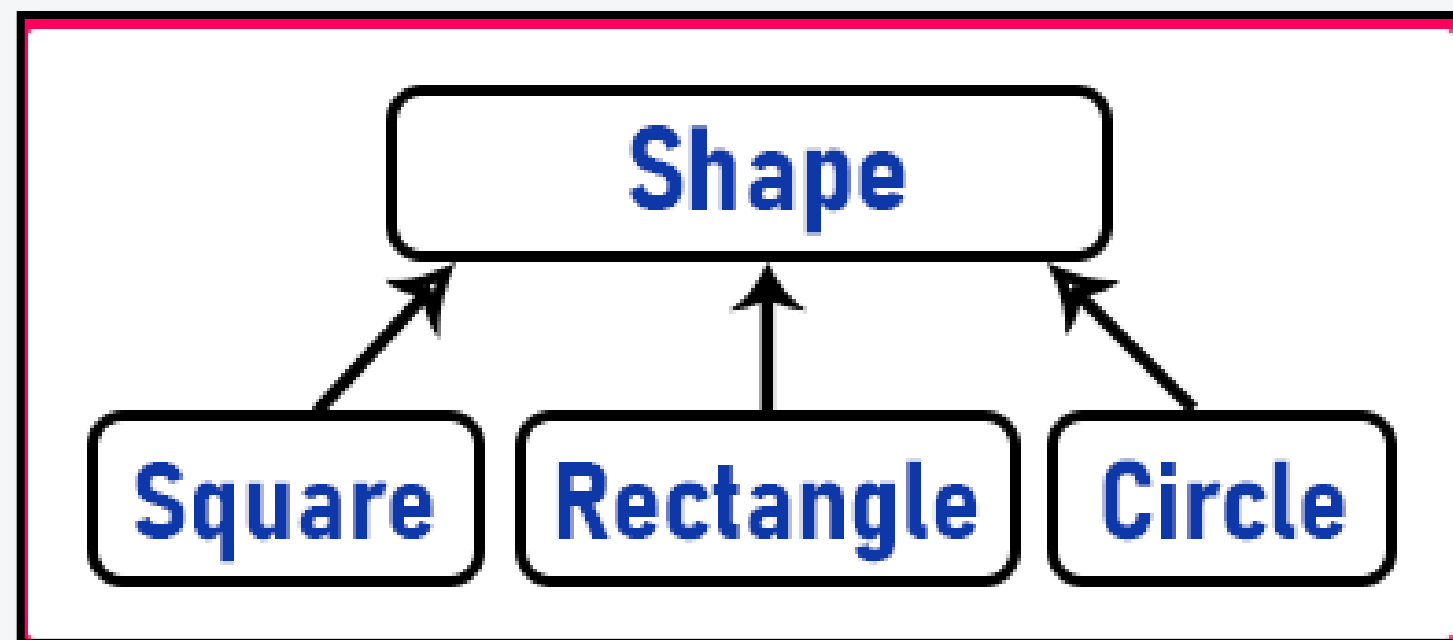


Multi Level Inheritance



```
class A {  
};  
class B : public A {  
};  
class C : public B {  
};
```

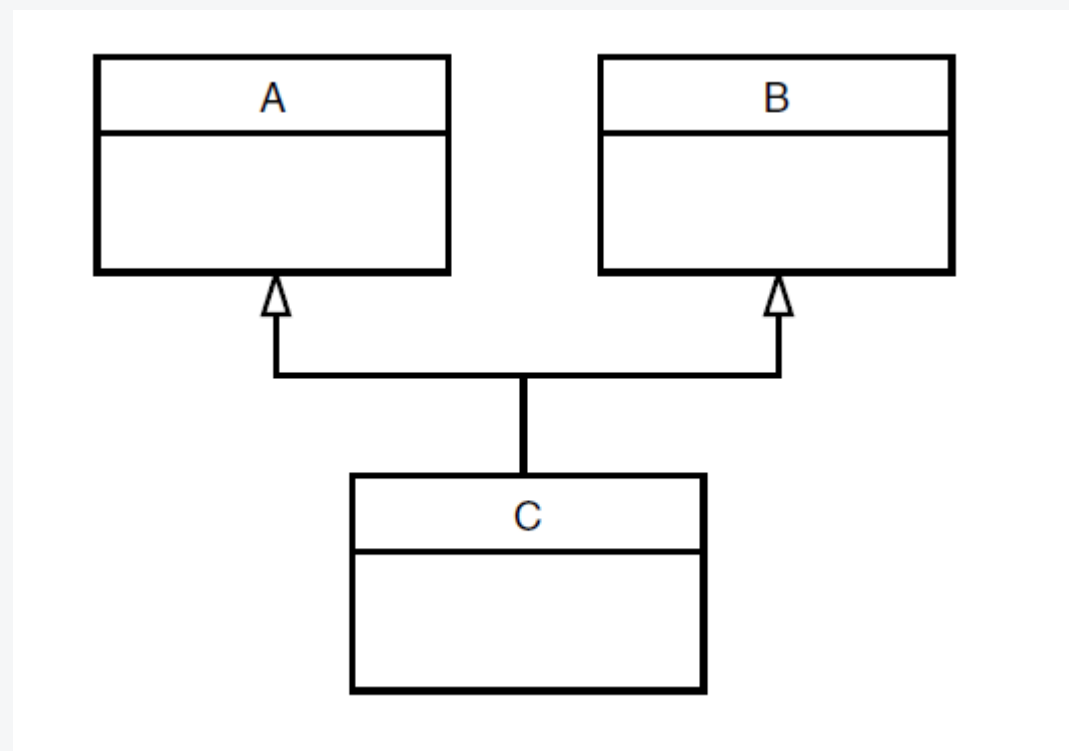

Hierarchical Inheritance



```
class BaseClass {};  
class Derived1 : public BaseClass {};  
class Derived2 : public BaseClass {};
```

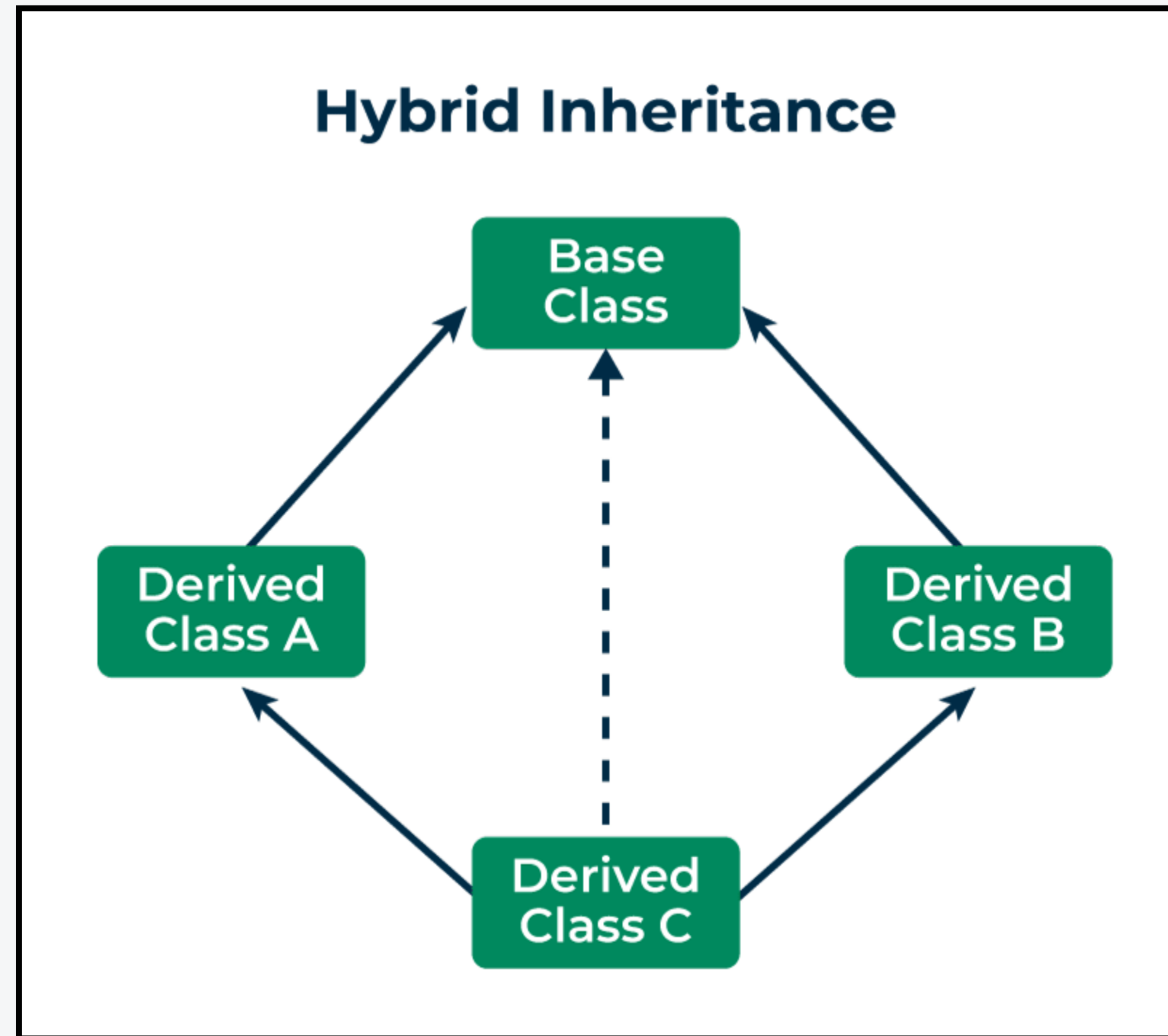
Multiple Inheritance

- A class can be derived from more than one base class. This is called multiple inheritance.

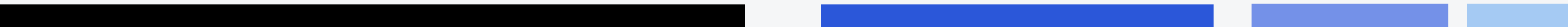


```
class A                                // base class A
{
};
class B                                // base class B
{
};
class C : public A, public B           // C is derived from A and B
{
};
```

Hybrid Inheritance



Multiple Inheritance: Complexity



Member Functions in Multiple Inheritance

```
#include <iostream>
using namespace std;
class Camera {
public:
    void takePhoto() {cout << "Camera: Capturing a photo..." << endl;}
    void showInfo() {cout << "Camera: 48 MP lens, 4K video recording" << endl;}
};
class Phone {
public:
    void makeCall() {cout << "Phone: Calling your friend..." << endl;}
    void showInfo() {cout << "Phone: 5G network, dual SIM supported" << endl;}
};
```

Member Functions in Multiple Inheritance

```
class SmartPhone : public Camera, public Phone {  
public:  
    void browseInternet() {cout << "SmartPhone: Browsing the internet..." << endl;}  
    void showAllInfo() {  
        Camera::showInfo();  
        Phone::showInfo();  
    }  
};
```

Member Functions in Multiple Inheritance

```
int main() {  
    SmartPhone myPhone;  
    cout << "=== Using my SmartPhone ===" << endl;  
    myPhone.takePhoto();  
    myPhone.makeCall();  
    myPhone.browseInternet();  
    cout << "\n=== Device Information ===" << endl;  
    myPhone.showAllInfo();  
    cout << "\n=== Direct calls using scope resolution ===" << endl;  
    myPhone.Camera::showInfo();  
    myPhone.Phone::showInfo();  
    return 0;  
}
```

Constructors in Multiple Inheritance

```
class Camera {  
protected:  
    int mP;  
public:  
    Camera():mP(12) {cout << "[Camera] Default cons. MP = " << mP << endl;}  
    Camera(int m):mP(m) { cout << "[Camera] Parameterized cons. MP = " << mP << endl; }  
    ~Camera() { cout << "[Camera] Destructor called." << endl;}  
    void showCamera() {cout << "Camera: " << mP << " MP lens" << endl;}  
};
```


Constructors in Multiple Inheritance

```
class Phone {  
protected:  
    string network;  
public:  
    Phone() : network("4G") {cout << "[Phone] Default cons. Network = " << network << endl;}  
    Phone(string net) : network(net) {cout << "[Phone] Param. Cons. called. Network = " <<  
        network << endl;}  
    ~Phone() {cout << "[Phone] Destructor called." << endl;}  
    void showPhone() {cout << "Phone: " << network << " network supported" << endl;}  
};
```

Constructors in Multiple Inheritance

```
class SmartPhone : public Phone, public Camera {
    string brand;
public:
    SmartPhone():brand("Unknown") {cout<<"[SmartPhone] Default cons. Brand = " << brand << endl;}
    SmartPhone(int mp, string net, string b): Phone(net), Camera(mp), brand(b)
    {cout << "[SmartPhone] Parameterized const. called. Brand = " << brand << endl;}
    ~SmartPhone() {cout << "[SmartPhone] Destructor called." << endl;}
    void showAll() {
        cout << "\n--- Device Info ---" << endl;
        cout << "Brand: " << brand << endl;
        showCamera();
        showPhone();}
};
```

Constructors in Multiple Inheritance

```
int main() {
    cout << "== Creating SmartPhone with DEFAULT constructor ==" << endl;
    {
        SmartPhone sp1;
        sp1.showAll();
        cout << "\n-- End of scope: destructors will be called in reverse order --" << endl;
    }

    cout << "\n== Creating SmartPhone with PARAMETERIZED constructor ==" << endl;
    {
        SmartPhone sp2(48, "5G", "Pixel");
        sp2.showAll();
        cout << "\n-- End of scope: destructors will be called in reverse order --" << endl;
    }

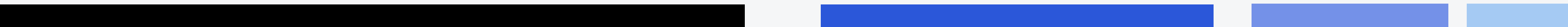
    return 0;
}
```


Ambiguity in Multiple Inheritance

Two base classes have functions with the same name, while a class derived from both base classes has no function with this name.

```
#include <iostream>
using namespace std;
/////////////////////////////////////////////////////////////////
class A
{
public:
    void show() { cout << "Class A\n"; }
};
class B
{
public:
    void show() { cout << "Class B\n"; }
};
class C : public A, public B
{
};
/////////////////////////////////////////////////////////////////
int main()
{
    C objC;           //object of class C
    // objC.show();    //ambiguous--will not compile
    objC.A::show();    //OK
    objC.B::show();    //OK
    return 0;
}
```

Hybrid Inheritance: Diamond Problem



Ambiguity in Hybrid Inheritance

Diamond Problem

```
#include <iostream>
using namespace std;
////////////////////////////////////
class A
{
public:
    void func();
};
class B : public A
{ };
class C : public A
{ };
class D : public B, public C
{ };
////////////////////////////////////
int main()
{
    D objD;
    objD.func(); //ambiguous: won't compile
    return 0;
}
```

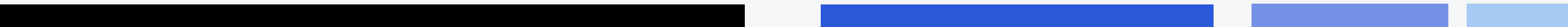
Ambiguity in Hybrid Inheritance

Diamond Problem

```
#include <iostream>
using namespace std;
class A{public:
void func() {cout << "Function in A\n";}
};
class B : virtual public A{};
class C : virtual public A{};
class D : public B, public C{ };

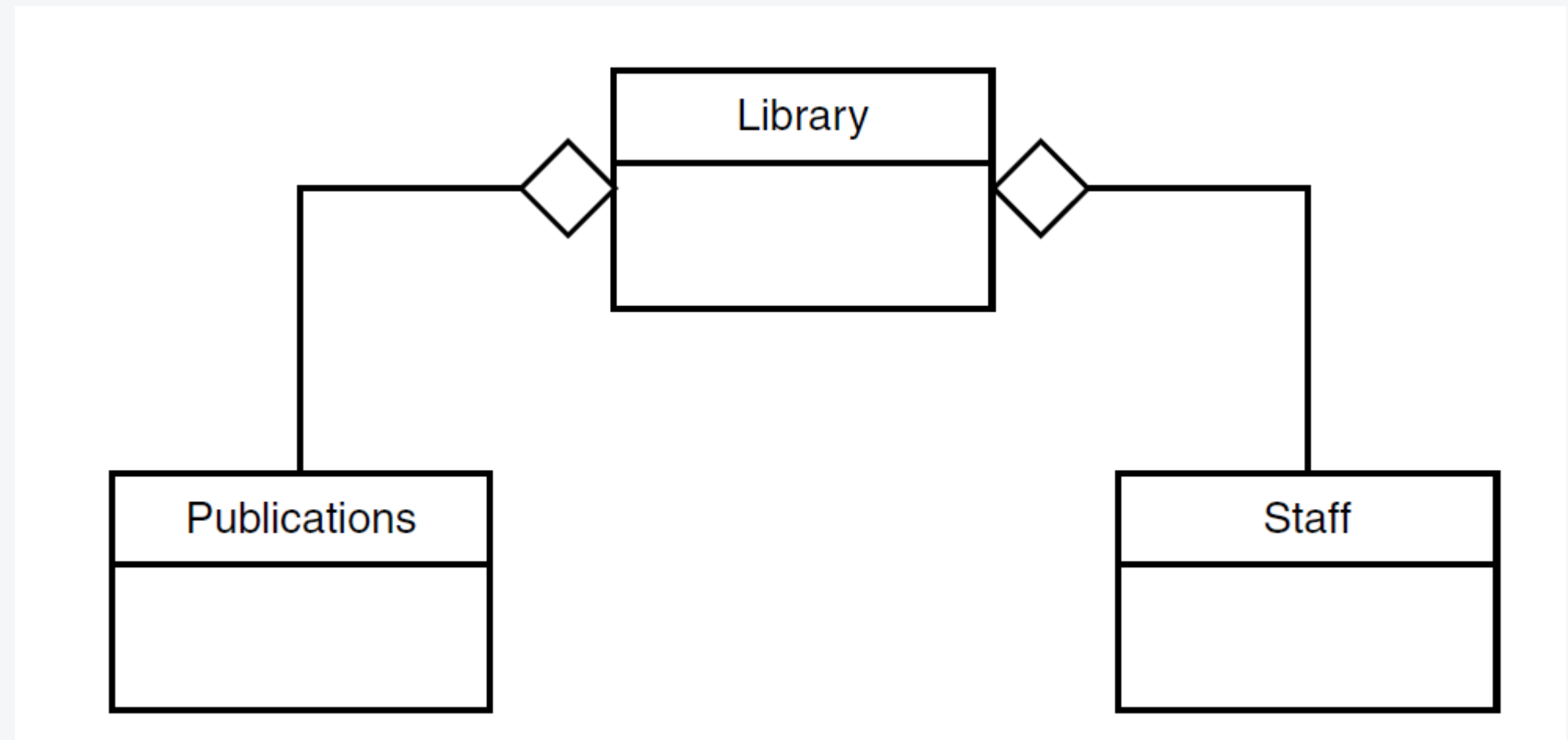
int main(){
D objD;
objD.func();
return 0;}
```

Classes Relation: Aggregation & Composition



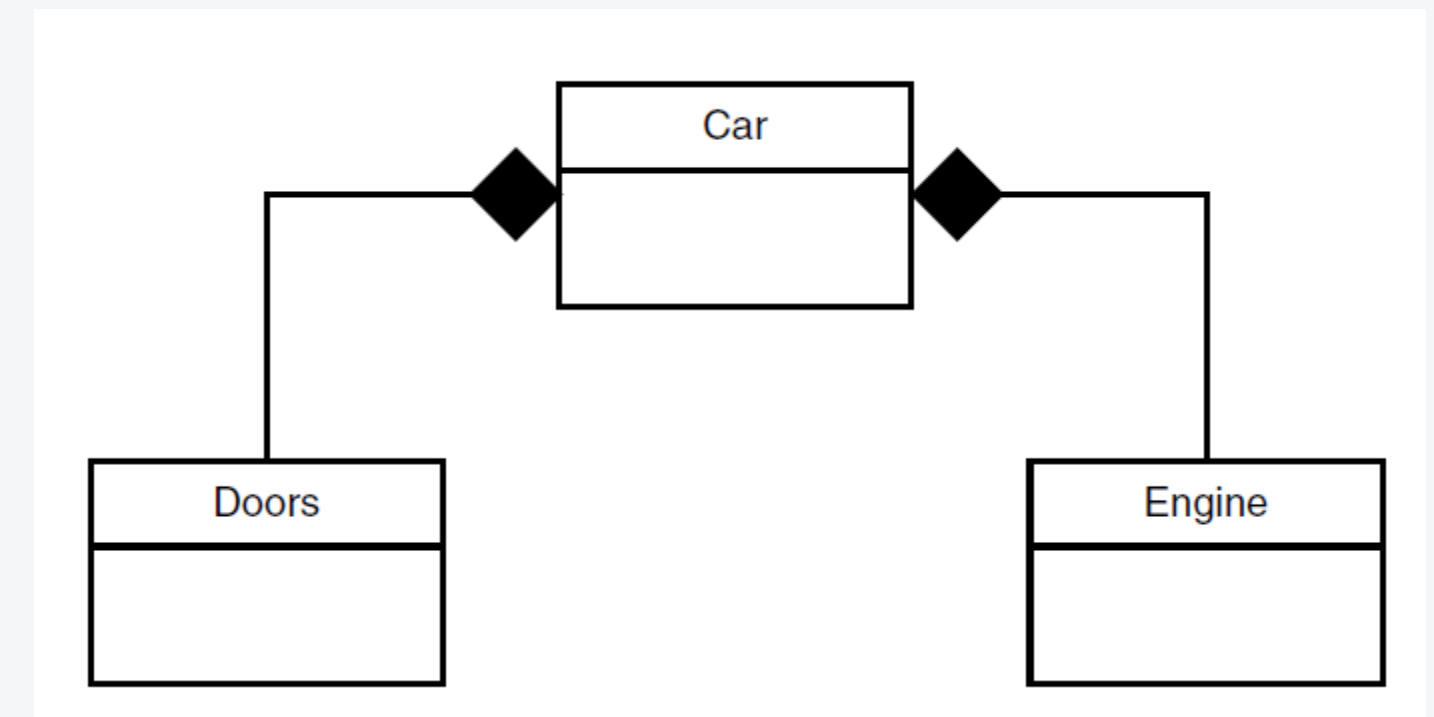
Aggregation

- A class has another class object but does not control its lifetime.
- Aggregation is called a “has a” relationship.
- In object-oriented programming, aggregation may occur when one object is an attribute of another.



Composition: A Stronger Aggregation

- Composition is a stronger form of aggregation.
- The lifetime of the part is the same as the lifetime of the whole.
- Composition is a “consists of” relationship.



1. ASSOCIATION — "uses-a

Two independent objects interact. Neither owns the other.



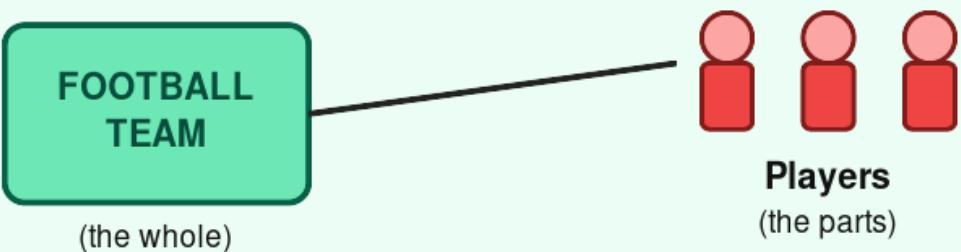
A Driver drives a Car.

Both can exist without each other —
the driver can walk away, the car can be driven by someone else.

Notation: A — B

2. AGGREGATION — weak "has-a

Whole contains parts, but parts can live independently.



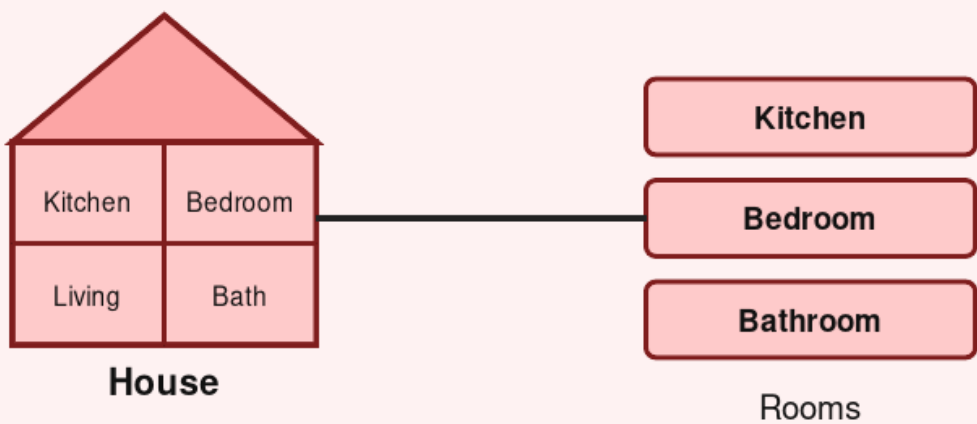
A Team has Players.

If the team is dissolved, the players still exist —
they can transfer and join another team.

Notation: A ◇ B (hollow diamond on whole)

3. COMPOSITION — strong "part-of

Parts cannot exist without the whole. Destroy the whole → parts die too.



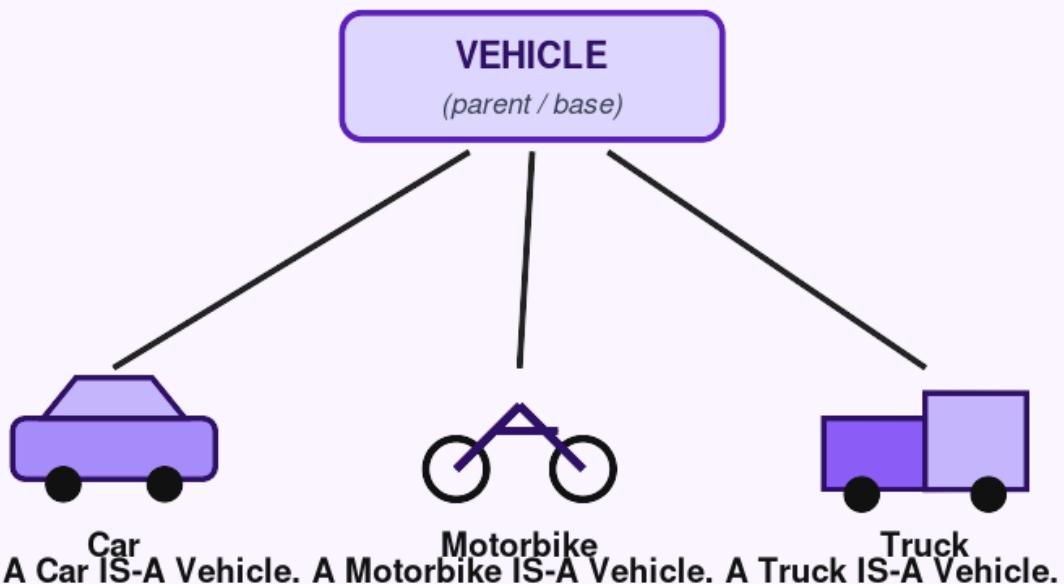
A House is composed of Rooms.

Demolish the house and the rooms cease to exist.

Notation: A ◆ B (filled diamond on whole)

4. INHERITANCE — "is-a

A child class is a specialised type of its parent.



Notation: Child — Parent (hollow triangle on parent)

Aggregation & Composition

```
#include <iostream>
using namespace std;
class Engine {
public:
    Engine(){
        cout<<"Engine created\n";
    }
    void start() {
        cout<<"Engine started\n";
    }
    ~Engine(){
        cout<<"Engine destroyed\n";
    }
};
```

```
class CarAggregation {
private: Engine* engine;
public:
    CarAggregation(Engine* e) {
        engine = e;
        cout<<"Car Agg. created\n";
    }
    void drive() {
        engine->start();
        cout << "Car Agg. driving\n";
    }
    ~CarAggregation() {
        cout<<"Car Agg. destroyed\n";
    }
};
```

```
CarComposition {
private: Engine engine;
public:
    CarComposition() {
        cout<<"Car Comp. created\n";
    }
    void drive() {
        engine.start();
        cout<<"Car Comp. driving\n";
    }
    ~CarComposition() {
        cou<<"Car Comp. destroyed\n";
    }
};
```

Aggregation & Composition

```
int main() {  
    cout << "\n--- Aggregation Example ---\n";  
    Engine e1;  
    CarAggregation carA(&e1);  
    carA.drive();  
    cout << "End of aggregation block\n\n";  
    cout << "---- Composition Example ----\n";  
    CarComposition carC;  
    carC.drive();  
    cout << "End of composition block\n";  
    return 0;}  

```